Contents

Introduction

This document contains commonly used code for competitive programming in c++ as preparation for the 2026 nordic-baltic olympiad of informatics.

Basics

Vim config

Here is a basic vim config for competitive programming in rust:

```
let mapleader = " "
nnoremap <leader>ff :%!rustfmt<cr>
nnoremap <leader>fm :Explore<cr>
tnoremap <Esc> <C-\><C-n>

syntax on
set clipboard=unnamedplus
set mouse=a
set hlsearch
set incsearch
set ignorecase
set smartcase

set number
set relativenumber
set cursorline
set nowrap

set tabstop=4
set shiftwidth=4
set expandtab
set autoindent
set smartindent

hi CursorLine cterm=NONE ctermbg=236
hi MatchParen ctermfg=0 ctermbg=188
```

Place this in `~/.vimrc` or `~/.config/nvim/init.vim` at the start of the competition.

C++ Boilerplate

```
#include <iostream>
using namespace std;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);
}
```

Note

Some environments provide `#include <bits/stdc++.h>`, but it is not always present. This non-standard header file imports all the standard libraries and avoids having to import many things in the same file.

Compiling the program

```
g++ -DEVAL -std=gnu++17 -O2 -pipe -o main main.cpp
```

Reading input values

Reading a single value:

```
int n;
cin >> n;
```

Note

This reads until the first whitespace character. To read the entire line, use

```
string s;
cin >> s;
```

Reading multiple values on the same line. Types can also be mixed in the same line, for example

```
int u, v;
Permission k;
cin >> u >> v >> k;
```

Reading values from multiple lines:

```
vector<int> xs(n);
for (int i = 0; i < n; i++) {
  int x;
  cin >> x;
  xs[i] = x;
}
```

Reading n values from the same line is best done using a stringstream:

```
#include <sstream>
#include <iostream>

string line;
getline(cin, line);
stringstream ss(line);
vector<int> xs;
int x;
while (ss >> x) {
  xs.push_back(x);
}
```

Alternatively, if the number of values is known (which applies to most problems), it can be simplified to avoid using a stringstream:

```
#include <iostream>
#include <vector>

int n;
cin >> n;
vector<int> xs;
```

```
while (n--) {
  int x;
  cin >> x;
  xs.push_back(x);
}
```

Binary

Parsing a binary value:

```
#include <string>

string input;
cin >> input;
int x = stoi(input, nullptr, 2);
```

Note

The radix 2 can also be replaced with any other integer.

Alternatively, a bitset can be used, as described in that section of this document.

Converting an integer to a binary string is usually done using a bitset:

```
int n = 42;
string s = bitset<10>(n).to_string();
```

In `c++20`, `format` can also be used:

```
int n = 42;
string s = format("{:b}", n);
```

Hex and octal

C++ has built in stream manipulators to convert between them.

```
#include <sstream>
#include <iomanip>

stringstream ss1;
ss1 << hex << 255;
string s = ss1.str(); // "ff"

stringstream ss2;
ss2 << oct << 123;
string s = ss2.str(); // "173"

stringstream ss3;
ss3 << uppercase << hex << 255;
string s = ss3.str(); // "FF"

stringstream ss4;
ss4 << hex << 42 << " " << dec << 42;
string s = ss4.str(); // "2a 42"
```

Useful Data Types

Defining Types

New types can be defined with `using` as follows

```
using ll = long long;
```

Number Types

Signed integers:

Type	Bits
`char`	8
`short`	16
`int`	16 / 32
`long`	32
`long long`	32

Note

These can be prefixed with `unsigned` to use the unsigned variants.

Floating-point numbers:

- `float` - 32 bits, accurate to about 7 decimal digits.
- `double` - 64 bits, accurate to about 15-17 decimal digits.
- `long double` - 80 or 128 bits.

For competitive programming, it is often better to use `<cstdint>` to provide the excact desired size.

Type	Bits
`int8_t`	8
`int16_t`	16
`int32_t`	32
`int64_t`	32
`uint8_t`	8
`uint16_t`	16
`uint32_t`	32
`uint64_t`	32

Limits

Useful when for example setting max values in DP

```
#include <limits>

const int INF = std::numeric_limits<int>::max();
const size_t INF = std::numeric_limits<size_t>::max();
const uint64_t INF = std::numeric_limits<uint64_t>::max();
```

An alternative version which is not recommended in production code, but easier to use for CP, is:

```
#include <climits>
```

This provides

- `INT_MIN` / `INT_MAX`
- `LLONG_MIN` / `LLONG_MAX`
- `ULLONG_MIN` / `ULLONG_MAX`
- etc.

Tuples

```
#include <tuple>

pair<int, int> x = { 1, 3 };
x.first; // 1
x.second; // 3

// These are equivalent:
tuple<int, int, int> x(1, 2, 3);
tuple<int, int, int> x = make_tuple(1, 2, 3);
tuple<int, int, int> x = { 1, 2, 3 };

get<0>(x); // 1
get<2>(x); // 3
```

Tuples can be destructured using `auto`:

```
tuple<int, int, int> x = { 1, 2, 3 };
auto [a, b, c] = x; // a = 1, b = 2, c = 3
```

By default, the above code will create copies of the values. To instead use references to the same memory, `&` can be used.

```
auto &[a, b, c] = x; // a = 1, b = 2, c = 3
```

Strings

```
#include <string>

string a = "Hello";
string b = " World";
string c = a + b;
cout << c << endl; // "Hello World"
c.length(); // 11
c[0]; // 'H'
c += "!"; // c = "Hello World!"
c.find("lo"); // 3
c.find("abcde"); // string::npos
c.substr(0, 5); // "Hello"
c.clear(); // c = ""

to_string(42); // "42"
stoi("1234"); // 1234
```

Chars

Chars are essentially 8-bit integers.

```
char c = 'A';
cout << c << endl; // "A"
cout << (int)c << endl; // 65
c += 1;
cout << x << endl; // "B"
```

Vectors

Vectors are lists of items with variable length.

```
#include <vector>
std::vector<int> nums = {10, 20, 30};
nums.push_back(40);
```

It van be initialized with a length by adding an argument like this:

```
vector<vector<pair<int, int>>> graph(n);
for (int i = 0; i < m; i++) {
  int a, b, k;
  cin >> a >> b >> k;
  graph[a].push_back({b, k});
  graph[b].push_back({a, k});
}
```

A default value can be provided as such:

```
vector<int> xs(5, 0); // [0, 0, 0, 0, 0]
```

It can also be nested:

```
vector<vector<int>> dist(n, vector<int>(v + 1, INF));
```

This will create a vector of `n` vectors, each with `v+1` values initialized to `INF`.

Bitsets

Bitsets are essentially boolean arrays but optimized to use only a single bit per value and allowing the use of boolean operators. It is for example useful when dealing with bitmasks. They provide the following operations

Operator	Operation
`&`	AND
`|`	OR
`^`	XOR
`~`	NOT
`<<`	Left Shift
`>>`	Right Shift

And here are some useful functions:

Operator	Operation
`b.count()`	Return the number of `1` bits
`b.any()`	True if at least one bit is `1`
`b.all()`	True if all bits are `1`
`b.none()`	True if no bits are `1`
`b.size()`	Return the total number of bits
`b.to_string()`	Convert to a string of binary
`b.to_ulong()`	Convert to unsigned long
`b.to_ullong()`	Convert to unsigned long long
`b.set(i)`	Set the bit at `i` to `1`
`b.reset(i)`	Set the bit at `i` to `0`
`b.flip(i)`	Flip the bit at `i`
`b.[i]`	Access the bit at index `i`
`b.test(i)`	Access bit (with bounds check)

```
#include <bitset>

bitset<10> b; // 0000000000
b.set(2); // 0000000100
b[2]; // true
b[1]; // false
```

Here is an example from my solution to `nio23-finale-noekkelkort`.

```
#include <bitset>
#include <iostream>
#include <queue>
#include <tuple>
#include <vector>
using namespace std;

using Permission = bitset<30>;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m, k, t;
  cin >> n >> m >> k >> t;

  int j = 0;
  vector<tuple<int, int, Permission>> connections(t);
  vector<vector<pair<int, Permission>>> graph(n);
  for (int i = 0; i < m; i++) {
    int u, v;
```

```
    Permission k;
    cin >> u >> v >> k;

    graph[u].push_back({v, k});
    graph[v].push_back({u, k});

    if (j < t) {
      connections[i] = {u, v, k};
    }
    j++;
  }

  // O(t (n + m))
  for (const auto &[u, v, k] : connections) {
    deque<int> queue;
    vector<bool> visited(n);
    Permission permission = ~k;
    queue.push_back(u);
    visited[u] = true;

    while (!queue.empty()) {
      int a = queue.front();
      queue.pop_front();

      if (a == v) {
        cout << "0" << endl;
        goto valid;
      }

      for (const auto &[b, k] : graph[a]) {
        if (!visited[b] && (permission & k).any()) {
          queue.push_back(b);
          visited[b] = true;
        }
      }
    }

    cout << "1" << endl;
  valid:;
  }
}
```

Queues

`std::queue` provides a FIFO queue.

```
#include <queue>

deque<int> queue;
queue.push(2);
queue.push(20);
queue.push(15);
// queue = [2, 10, 15]
queue.front(); // 2
queue.pop();
queue.front(); // 10
queue.pop();
// queue = [15]
```

Note

Unlike in other languages like rust, `.pop()` cannot be used to both retrieve and remove an element from a queue. Instead, this has to be done in two operations.

Deque

`queue::deque` provides a double-ended queue.

```
deque<int> queue;
queue.push_back(2);
queue.push_back(5);
queue.push_front(10);
// queue = [10, 2, 5]
queue[1]; // 2
queue.front(); // 10
queue.pop_front();
queue.back(); // 5
queue.pop_back();
queue.front(); // 2
// queue = [2]
```

Priority Queue

By default, `queue::priority_queue` is a max-heap.

```
priority_queue<int> pq;
pq.push(10);
pq.push(30);
pq.push(20);
pq.push(5);
// pq = [30, 20, 10, 5]
```

A min-heap can be implemented by changing the arguments:

```
priority_queue<int, vector<int>, greater<int>> pq;
pq.push
pq.push(10);
pq.push(30);
pq.push(20);
pq.push(5);
// pq = [5, 10, 20, 30]
pq.top(); // 5
pq.pop(); // pq = [10, 20, 30]
```

In algorithms like Dijkstra, we usually want to have a priority queue of tuples.

```
priority_queue<tuple<int, int, int>,
               vector<tuple<int, int, int>>,
               greater<tuple<int, int, int>>> pq;

pq.push({10, 1, 5});
pq.push({5, 2, 3});
pq.push({5, 1, 2});
auto [cost, u, b] = pq.top();
// cost = 5, u = 1, b = 2
```

Algorithms

Graphs

Dijkstra

Dijkstra's algorithm is an algorithm for finding the shortest path in an undirected graph with positive edge weights. It runs in time $O(n \log n)$ (or more accurately, $O((V+E)\log V)$). It uses a priority queue to order nodes by the total weight. The algorithm is essentially an expansion of 0-1 BFS with a binary heap instead of a double ended queue. Here is a simple implementation:

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

const int INF = 1e9;

int main() {
  vector<vector<pair<int, int>>> graph(n);

  priority_queue<pair<int, int>, vector<pair<int, int>>,
                 greater<pair<int, int>>>
      pq;

  vector<int> dist(n, INF);
  dist[0] = 0;
  pq.push({0, 0});

  while (!pq.empty()) {
    auto [d, u] = pq.top();
    pq.pop();

    if (d > dist[u])
```

```
      continue;

    for (auto &[v, weight] : graph[u]) {
      int new_cost = d + weight;
      if (new_cost < dist[v]) {
        dist[v] = new_cost;
        pq.push({dist[v], v});
      }
    }
  }

  cerr << "[ ";
  for (int &i : dist)
    cerr << i << " ";
  cerr << "]" << endl;
}
```

Many problems require a modified version of dijkstra. Here is an example from `nio21-finale-togtur`, in which the queue and the DP table for distance has an added dimension for the number of free tickets available.

```
#include <iostream>
#include <limits>
#include <queue>
#include <tuple>
#include <vector>
using namespace std;

const int INF = std::numeric_limits<int>::max();

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int n, m, v;
  cin >> n >> m >> v;

  vector<vector<pair<int, int>>> graph(n);
  for (int i = 0; i < m; i++) {
    int a, b, k;
    cin >> a >> b >> k;
    graph[a].push_back({b, k});
    graph[b].push_back({a, k});
  }

  priority_queue<tuple<int, int, int>, vector<tuple<int, int,
int>>,
                 greater<tuple<int, int, int>>>
      pq;
  vector<vector<int>> dist(n, vector<int>(v + 1, INF));

  pq.push({0, 0, v});
  dist[0][v] = 0;

  while (!pq.empty()) {
    auto [c, u, b] = pq.top();
    pq.pop();

    if (c > dist[u][b]) {
      continue;
    }

    if (u == 1) {
      cout << c << endl;
      return 0;
    }

    for (const auto &[v, cost] : graph[u]) {
      int new_cost = c + cost;
      if (new_cost < dist[v][b]) {
        dist[v][b] = new_cost;
        pq.push({new_cost, v, b});
      }

      if (b > 0 && c < dist[v][b - 1]) {
        dist[v][b - 1] = c;
        pq.push({c, v, b - 1});
      }
    }
  }
}
```